CHEAT SHEET · GROUPE 1

Authentification & autorisation

AuthN vs AuthZ · OAuth2 · PKCE · JWT · FastAPI · OWASP | Sources : RFC 6749, RFC 7636, RFC 7519, OWASP API Top 10

AUTHN vs AUTHZ

Authentification	Autorisation
Qui es-tu ?	Que peux-tu faire ?
Login, mot de passe, MFA, token	Rôles, scopes, propriété
401 Unauthorized	403 Forbidden
OAuth2 / OIDC, sessions	RBAC, ABAC, policies

MÉCANISMES

Mécanisme	Usage	Limite
Basic	Interne, tests	TLS obligatoire
Clé API	Serveur à serveur	Rotation, fuite
Session	Web	CSRF, scaling
Bearer / JWT	API, mobile, SPA	Vol, expiration
OAuth2 / OIDC	Délégation, SSO	Configuration
mTLS	B2B, service mesh	Certificats

OAUTH2 : LES RÔLES

- **Resource Owner** : l'utilisateur propriétaire des données.
- **Client** : l'application qui demande l'accès.
- **Authorization Server** : authentifie et délivre les tokens.
- **Resource Server** : l'API qui valide le token et sert les données.

OIDC = OAuth2 + identité (ID token, `/userinfo`).

LES FLOWS

Flow	Contexte
Authorization Code + PKCE	SPA, mobile, web
Client Credentials	Serveur à serveur
Device Code	TV, CLI, IoT
Refresh Token	Renouveler l'access token
Implicit / Password	Dépréciés — à éviter

PKCE : SÉQUENCE

```
1. verifier = random(43..128)
   challenge = BASE64URL(SHA256(verifier))

2. GET /authorize
   ?response_type=code
   &client_id=...
   &redirect_uri=https://app/cb
   &scope=tasks:read tasks:write
   &state=...
   &code_challenge=...&code_challenge_method=S256

3. Redirection : ?code=...&state=...

4. POST /token
   grant_type=authorization_code
   &code=...&code_verifier=...
   &client_id=...&redirect_uri=...

5. API : Authorization: Bearer <access_token>
```

state = anti-CSRF. **PKCE** = anti-interception du code.

JWT : STRUCTURE

```
header.payload.signature

# Header
{"alg":"RS256","typ":"JWT","kid":"2026-10"}

# Payload (lisible par tous !)
{
  "iss":"https://auth.exemple.com",
  "sub":"user-42",
  "aud":"api-taches",
  "exp":1793520000,
  "iat":1793516400,
  "scope":"tasks:read tasks:write",
  "role":"user",
  "jti":"8f14e45f-..."
}
```

Signature = intégrité + authenticité, **pas** confidentialité.

JWT : VALIDATION

```
import jwt
from jwt import InvalidTokenError

def decode_token(token: str) -> dict:
    try:
        return jwt.decode(
            token, PUBLIC_KEY,
            algorithms=["RS256"],
            audience="api-taches",
            issuer="https://auth.exemple.com",
            options={"require":
                ["exp","iat","iss","aud"]},
        )
    except InvalidTokenError:
        raise HTTPException(401, "Token invalide")
```

- Vérifier `alg`, `exp`, `nbf`, `iss`, `aud`.
- Utiliser JWKS + `kid` pour les clés asymétriques.
- Refuser `alg=none` et les algorithmes non attendus.
- Limiter le *clock skew* (30–60 s).

FASTAPI + JWT

```
from fastapi.security import OAuth2PasswordBearer
oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="/auth/token"
)

def get_current_user(
    token: str = Depends(oauth2_scheme),
):
    payload = decode_token(token)
    user = users.get(payload["sub"])
    if not user:
        raise HTTPException(401, "Utilisateur inconnu")
    return user

@app.get("/me")
def me(user=Depends(get_current_user)):
    return user
```

BONNES PRATIQUES JWT

- Access token court (5–15 min) + refresh token.
- Rotation et révocation des refresh tokens.
- Cookies `HttpOnly` + `Secure` + `SameSite` pour le web.
- Pas de données sensibles dans le payload.
- Pas de token dans l'URL ni dans les logs.
- Toujours HTTPS.
- Clés séparées par environnement, rotation.

ERREURS OAUTH2

- Ne pas valider `state` → CSRF.
- Utiliser Implicit / Password → fuite de token.
- `redirect_uri` non validée → vol de code.
- Scopes trop larges → excès de privilèges.
- Tokens dans l'URL / les logs.

VULNÉRABILITÉS JWT

- `alg=none` accepté.
- Confusion HS256 / RS256.
- Audience non vérifiée.
- Expiration trop longue.
- Stockage `localStorage` → XSS.
- Pas de révocation possible.

OWASP API TOP 10 · 2023

API1	BOLA — propriété des objets
API2	Broken Authentication
API3	Property Level Authorization
API4	Resource Consumption
API5	Function Level Authorization
API6	Sensitive Business Flows
API7	SSRF
API8	Security Misconfiguration
API9	Inventory Management
API10	Unsafe Consumption of APIs

CHECKLIST SÉCURITÉ

- HTTPS partout, HSTS.
- AuthN + AuthZ sur chaque endpoint.
- Vérifier la propriété (BOLA) et les rôles.
- Valider entrées, tailles, formats.
- Rate limiting + quotas.
- Secrets hors code, rotation.
- Logs d'audit sans secrets.
- Tests d'intrusion, `pip-audit`.

CYCLE DE VIE DES TOKENS

- **Access token :** 5–15 min, porté par le client.
- **Refresh token :** plus long, rotation à chaque usage.
- **Révocation :** logout, changement de mot de passe, incident.
- **Stockage web :** cookie `HttpOnly` + `Secure` + `SameSite`.
- **Monitoring :** détecter les réutilisations de refresh token.

SESSION VS JWT

Session	JWT
État serveur	Sans état
Révocation facile	Révocation complexe
Cookie sécurisé	Bearer token
Scaling + store	Scaling simple
Web classique	API, mobile, services

FASTAPI : LOGIN & TOKEN (EXEMPLE)

```
from fastapi.security import OAuth2PasswordRequestForm

@app.post("/auth/token")
def login(form: OAuth2PasswordRequestForm = Depends()):
    user = authenticate(form.username, form.password)
    if not user:
        raise HTTPException(401, "Identifiants invalides")
    access = create_access_token(
        sub=user.id, scopes=["tasks:read", "tasks:write"],
        expires_minutes=15,
    )
    refresh = create_refresh_token(sub=user.id)
    return {"access_token": access, "refresh_token": refresh,
            "token_type": "bearer", "expires_in": 900}

@app.post("/auth/refresh")
def refresh(refresh_token: str):
    payload = decode_refresh(refresh_token)
    return {"access_token": create_access_token(sub=payload["sub"])}
```


RFC 6749 · RFC 7636 · RFC 7519 · OWASP API Security Top 10 2023